Application-specific access policies using Zscaler Private Access (ZPA)

TEAM MEMBERS DETAILS

TEAM NO: 3

TEAM ID: 69a678eb8def0a9d6fb4c542

TEAM LEADER: ANUSHRI.V

COLLEGE NAME: RATHINAM COLLEGE OF LIBERAL ARTS AND SCIENCE @ TIPS GOLBAL

COLLEGE CODE: CD

LIST OF TEAMMMATES:

S.NO	NAME	COLLEGE	CONTACT
1	ANUSHRI.V	RATHINAM COLLEGE OF LIBERAL ARTS AND SCIENCE @ TIPS GOLBAL	Anushri.bcs23t@rathinam.in
2	BALA.M	RATHINAM COLLEGE OF LIBERAL ARTS AND SCIENCE @ TIPS GOLBAL	yashikamuthaiah@gmail.com
3	PRIYA DHARSHINI. M	RATHINAM COLLEGE OF LIBERAL ARTS AND SCIENCE @ TIPS GOLBAL	priyadhrashinisreemanth@ gmail.com
4	SAKTHIVEL	RATHINAM COLLEGE OF LIBERAL ARTS AND SCIENCE @ TIPS GOLBAL	Saktthi1@gmail.com
5	NANDHI GOPAL	RATHINAM COLLEGE OF LIBERAL ARTS AND SCIENCE @ TIPS GOLBAL	nandhinigopal@gmail.com

ABSTRACT

With the rapid growth of cloud computing and remote work environments, organizations require secure and efficient methods to control access to internal applications. Traditional network security models often provide broad network access, which increases the risk of unauthorized entry and cyber threats. Application-specific access policies offer a more secure approach by granting users access only to the particular applications they need.

This project focuses on implementing application-specific access policies using **Zscaler Private Access (ZPA)**, a cloud-based Zero Trust Network Access (ZTNA) solution. ZPA enables secure access to private applications without exposing them directly to the internet. Instead of granting full network access, the system verifies user identity, device posture, and contextual information before allowing access to specific applications.

The proposed approach ensures improved security, reduced attack surface, and better control over application usage. By enforcing granular access policies, organizations can protect sensitive resources while maintaining seamless connectivity for authorized users. This model supports modern enterprise environments by enabling secure remote access, simplifying network architecture, and aligning with the Zero Trust security framework.

Overall, the use of application-specific access policies through ZPA enhances organizational security, improves access management, and provides a scalable solution for protecting private applications in distributed and cloud-based infrastructures.

USE CASE SCENARIOUS

1. Employee Securely Accessing Internal HR Application

Scenario:

A company employee working from home wants to access the internal HR portal. The organization uses **Zscaler Private Access (ZPA)** to ensure that only authorized employees can access the application.

Process:

The employee logs into the company system using corporate credentials.

ZPA verifies the user identity through authentication.

The system checks the access policy configured for the HR application.

If the employee belongs to the HR department or authorized group, access is granted.

Expected Output:

User successfully connects to the HR portal.

Only the HR application becomes accessible.

Other internal applications remain hidden and inaccessible.

2. Unauthorized User Attempting Access

Scenario:
A user from another department tries to access the internal finance application without permission.

Process:

The user logs in through the secure gateway.

ZPA checks the application access policy.

The system identifies that the user does not have permission for the finance application.

Expected Output:

Access to the finance application is denied.

The user receives a “Permission Denied” message.

The attempt is logged for security monitoring

3. Device Security Verification

Scenario:
An employee tries to access a sensitive internal application using a personal device that does not meet company security requirements.

Process:

The user attempts to connect to the application.

ZPA checks device posture (antivirus status, OS updates, security compliance).

The device fails to meet security policy requirements.

Expected Output:

Access to the application is blocked.

The user receives a notification indicating the device is not compliant.

The event is recorded in the security logs.

4. Secure Access from Different Locations

Scenario:
An employee travels and tries to access internal applications from another country.

Process:

The user logs in using secure authentication.

ZPA verifies identity and location context.

Access policies evaluate risk factors.

Expected Output:

If location is allowed → access granted to specific applications only.

If location is restricted → access denied or additional verification required.

5. Administrator Updating Access Policies

Scenario:
The IT administrator updates application access policies to allow only managers to access a confidential project portal.

Process:

Administrator logs into the ZPA management console.

Updates access control policies for the project application.

Saves and applies the new policy.

Expected Output:

Only users with the “Manager” role can access the project portal.

Other users cannot even see the application in the network.

Policy changes are applied immediately across the system

STAGE 1

TITLE:

Application-specific access policies using Zscaler Private Access (ZPA)

OVERVIEW:

Application-Specific Access Policies using Zscaler Private Access (ZPA) is a modern security approach designed to provide secure and controlled access to internal applications without exposing the entire network. Traditional security models, such as VPNs, allow users to connect to the whole corporate network once authenticated, which increases the risk of unauthorized access and cyber threats. In contrast, ZPA follows the Zero Trust principle, where users are granted access only to specific applications based on defined security policies.

In this system, access is granted after verifying multiple factors such as user identity, device security status, location, and organizational role. Instead of connecting users directly to the network, ZPA creates a secure connection between the user and the requested application. This approach ensures that internal applications remain hidden from the public internet and are accessible only to authorized users.

The use of application-specific access policies helps organizations improve security, reduce the attack surface, and protect sensitive business data. It also enables employees to securely access company applications from any location, supporting remote work and cloud-based infrastructures. Overall, ZPA provides a scalable and efficient solution for secure application access while maintaining strong access control and monitoring capabilities.

LIST OF VULNERABILITY :

S NO	CATEGORY NAME	VULNERABILITY NAME	CWE/CVE-NO
1	Broken Access Control	Relative path traversal	CWE-23
2	Security Misconfiguration	Active debug code	CWE-489
3	Software Supply Chain Failures	Use of obsolaete function	CWE-447
4	Cryptographic Failures	Weak encoding for password	CWE-261
5	Injection	Improper input validation	CWE-20
6	Insecure Design	External Control of fFle Name or path	CWE-73
7	Authentication Failures	Use of hard coded password	Cwe-259
8	Software or Data Integrity Failures	Usetrusted Search path	CWE-426
9	Security Logging & Alerting Failures	Information loss of omission	CWE-532
10	Mishandling of Exceptional Conditions	Unchecked return value	CWE252

1. **VULNERABILITY NAME:**

Relative Path Traversal

CWE:

CWE-23

OWASP/SANS CATEGORY:

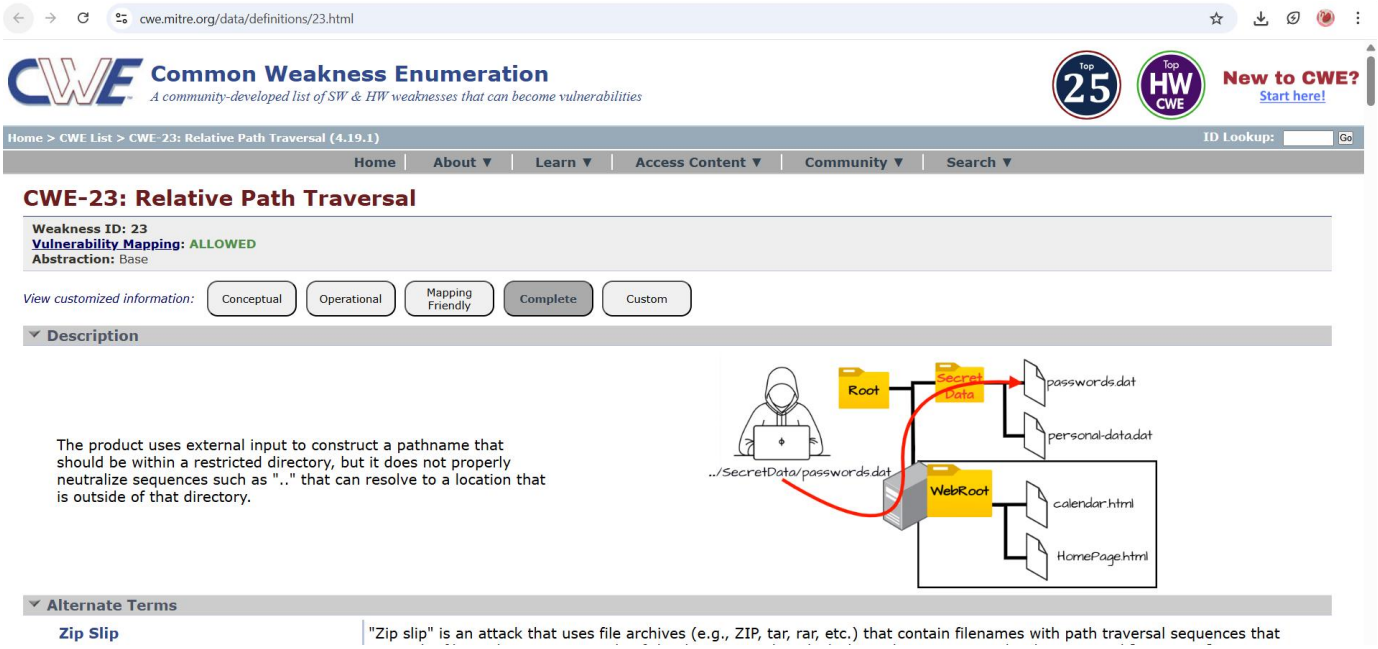
OWASP Top 10 – A01: Broken Access Control

DESCRIPTION:

Relative path traversal occurs when an application allows users to access files or directories outside the intended directory structure by manipulating file paths. Attackers may use special characters such as "../" to navigate to restricted directories.

BUSINESS IMPACT:

Attackers may gain unauthorized access to sensitive files, configuration data, or system resources, leading to information leakage and potential system compromise.



2.VULNERABILITY NAME:

Active Debug Code

CWE:

CWE-489

OWASP/SANS CATEGORY:

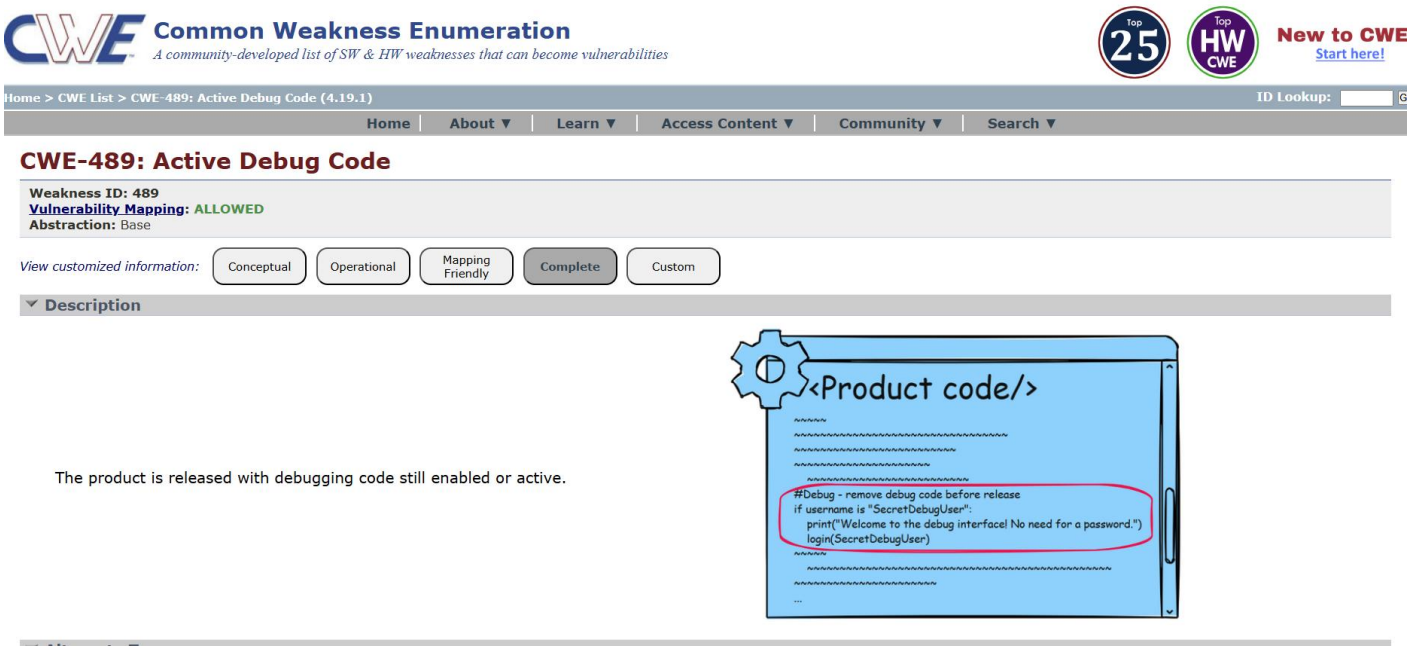
OWASP Top 10 – A05: Security Misconfiguration

DESCRIPTION:

Active debug code refers to debugging features that remain enabled in a production environment. These debugging mechanisms may reveal sensitive system information or allow unauthorized actions.

BUSINESS IMPACT:

Attackers may exploit debug functionality to access confidential system information, bypass security controls, or manipulate application behavior.



3.VULNERABILITY NAME:

Use of Obsolete Function

CWE:

CWE-477

OWASP/SANS CATEGORY:

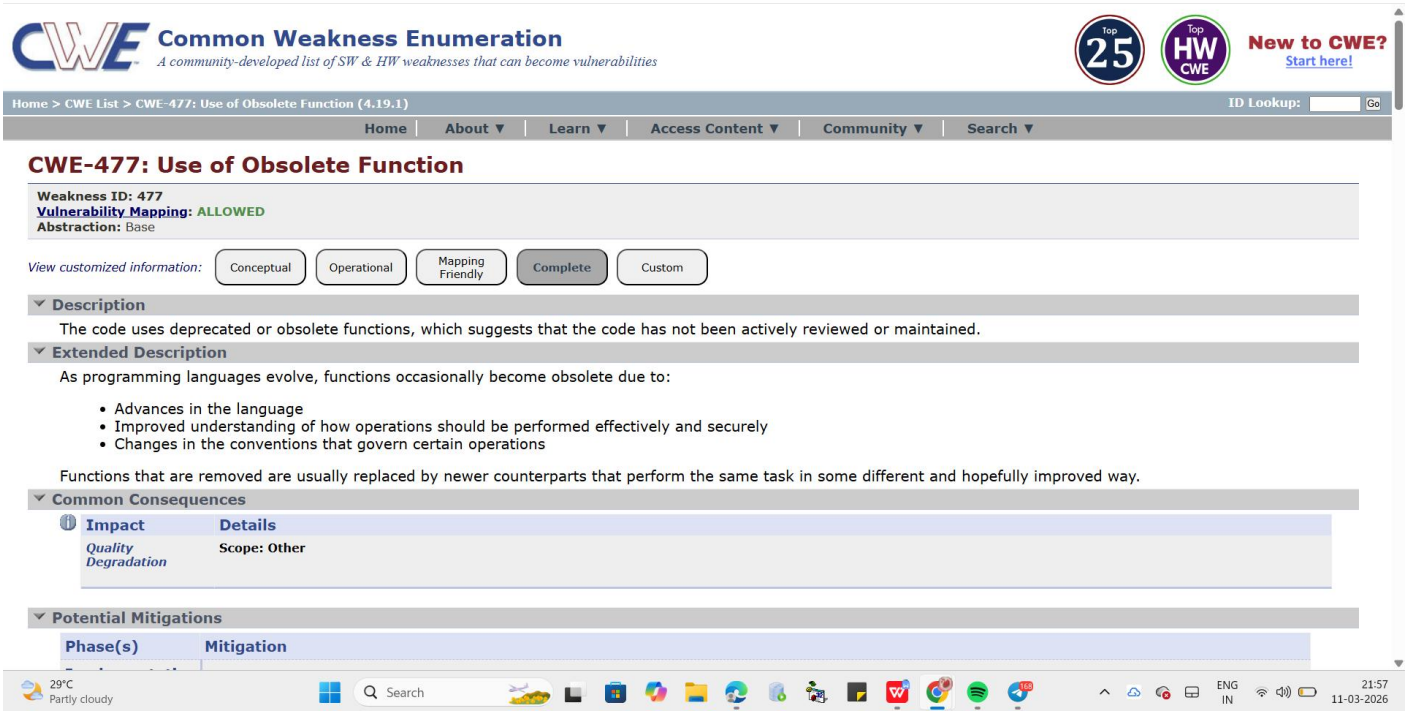
OWASP Top 10 – A06: Vulnerable and Outdated Components

DESCRIPTION:

This vulnerability occurs when outdated or deprecated functions are used in the software. Such functions may contain known security weaknesses and may not be supported by modern security updates.

BUSINESS IMPACT:

Using obsolete functions may expose the system to known exploits, resulting in security breaches, data loss, or application instability.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-477: Use of Obsolete Function (4.19.1) | ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-477: Use of Obsolete Function

Weakness ID: 477
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The code uses deprecated or obsolete functions, which suggests that the code has not been actively reviewed or maintained.

Extended Description

As programming languages evolve, functions occasionally become obsolete due to:

- Advances in the language
- Improved understanding of how operations should be performed effectively and securely
- Changes in the conventions that govern certain operations

Functions that are removed are usually replaced by newer counterparts that perform the same task in some different and hopefully improved way.

Common Consequences

Impact	Details
Quality Degradation	Scope: Other

Potential Mitigations

Phase(s)	Mitigation

29°C Partly cloudy | Search | ENG IN | 21:57 11-03-2026

4.VULNERABILITY NAME:

Weak Encoding for Password

CWE:

CWE-261

OWASP/SANS CATEGORY:

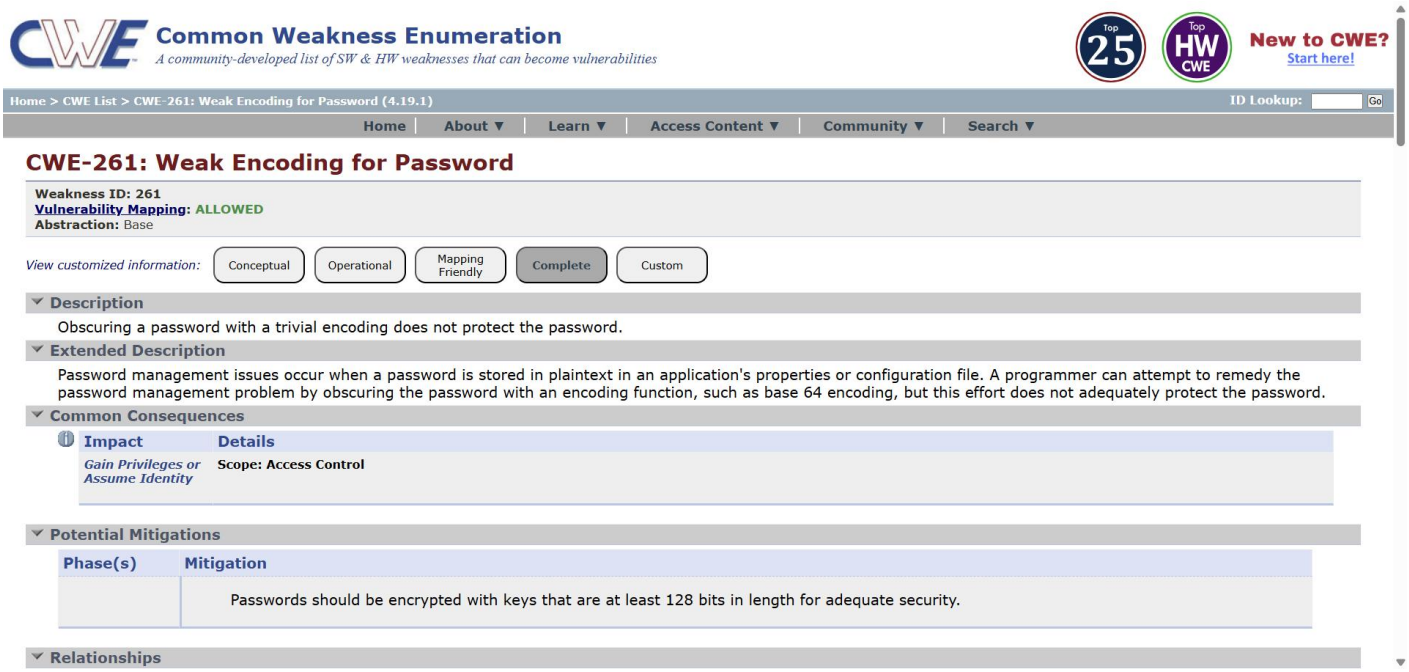
OWASP Top 10 – A02: Cryptographic Failures

DESCRIPTION:

Weak encoding or improper encryption of passwords allows attackers to easily decode or guess stored credentials. Poor cryptographic practices weaken the security of authentication mechanisms.

BUSINESS IMPACT:

Compromised passwords may allow attackers to gain unauthorized access to user accounts and sensitive systems.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-261: Weak Encoding for Password (4.19.1) | ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-261: Weak Encoding for Password

Weakness ID: 261
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

Obscuring a password with a trivial encoding does not protect the password.

Extended Description

Password management issues occur when a password is stored in plaintext in an application's properties or configuration file. A programmer can attempt to remedy the password management problem by obscuring the password with an encoding function, such as base 64 encoding, but this effort does not adequately protect the password.

Common Consequences

Impact	Details
Gain Privileges or Assume Identity	**Scope: Access Control**

Potential Mitigations

Phase(s)	Mitigation
	Passwords should be encrypted with keys that are at least 128 bits in length for adequate security.

Relationships

5.VULNERABILITY NAME:

Improper Input Validation

CWE:

CWE-20

OWASP/SANS CATEGORY:

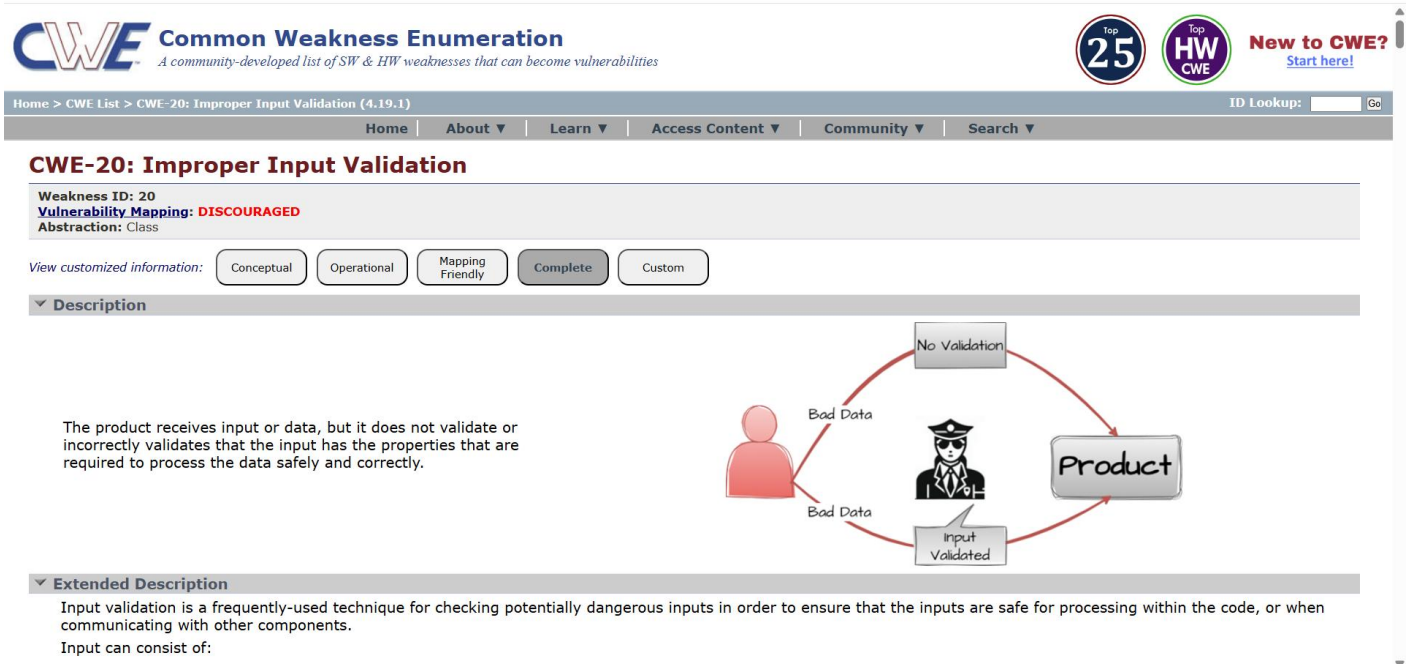
OWASP Top 10 – A03: Injection

DESCRIPTION:

Improper input validation occurs when an application fails to properly validate or sanitize user input. Attackers can inject malicious data or commands into the system.

BUSINESS IMPACT:

This vulnerability may lead to unauthorized system access, data manipulation, application crashes, or execution of malicious commands.



6.VULNERABILITY NAME:

External Control of File Name or Path

CWE:

CWE-73

OWASP/SANS CATEGORY:

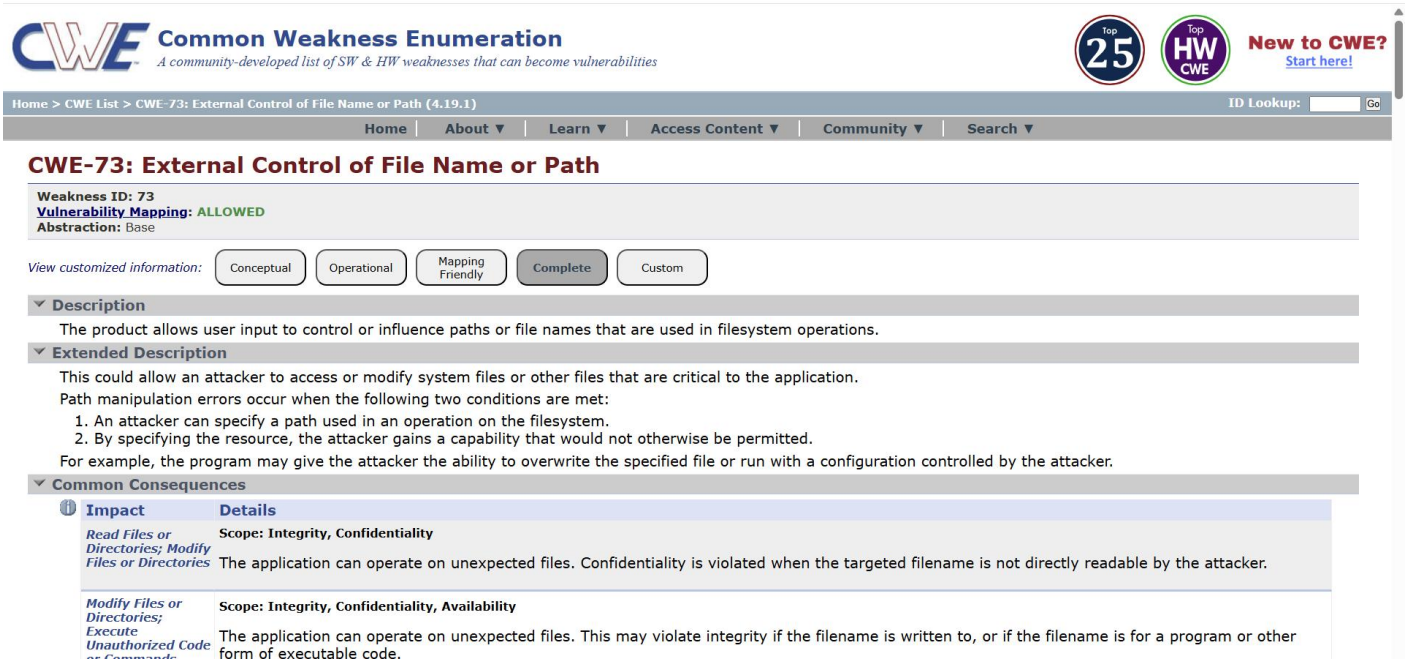
OWASP Top 10 – A04: Insecure Design

DESCRIPTION:

This vulnerability occurs when an application allows external input to control file names or file paths without proper validation. Attackers may manipulate these inputs to access unauthorized files.

BUSINESS IMPACT:

Sensitive files may be accessed or modified, leading to data leakage or system compromise.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-73: External Control of File Name or Path (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-73: External Control of File Name or Path

Weakness ID: 73
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product allows user input to control or influence paths or file names that are used in filesystem operations.

Extended Description

This could allow an attacker to access or modify system files or other files that are critical to the application.

Path manipulation errors occur when the following two conditions are met:

1. An attacker can specify a path used in an operation on the filesystem.
2. By specifying the resource, the attacker gains a capability that would not otherwise be permitted.

For example, the program may give the attacker the ability to overwrite the specified file or run with a configuration controlled by the attacker.

Common Consequences

Impact	Details
Read Files or Directories; Modify Files or Directories	**Scope: Integrity, Confidentiality** The application can operate on unexpected files. Confidentiality is violated when the targeted filename is not directly readable by the attacker.
Modify Files or Directories; Execute Unauthorized Code or Commands	**Scope: Integrity, Confidentiality, Availability** The application can operate on unexpected files. This may violate integrity if the filename is written to, or if the filename is for a program or other form of executable code.

7.VULNERABILITY NAME:

Use of Hard-Coded Password

CWE:

CWE-259

OWASP/SANS CATEGORY:

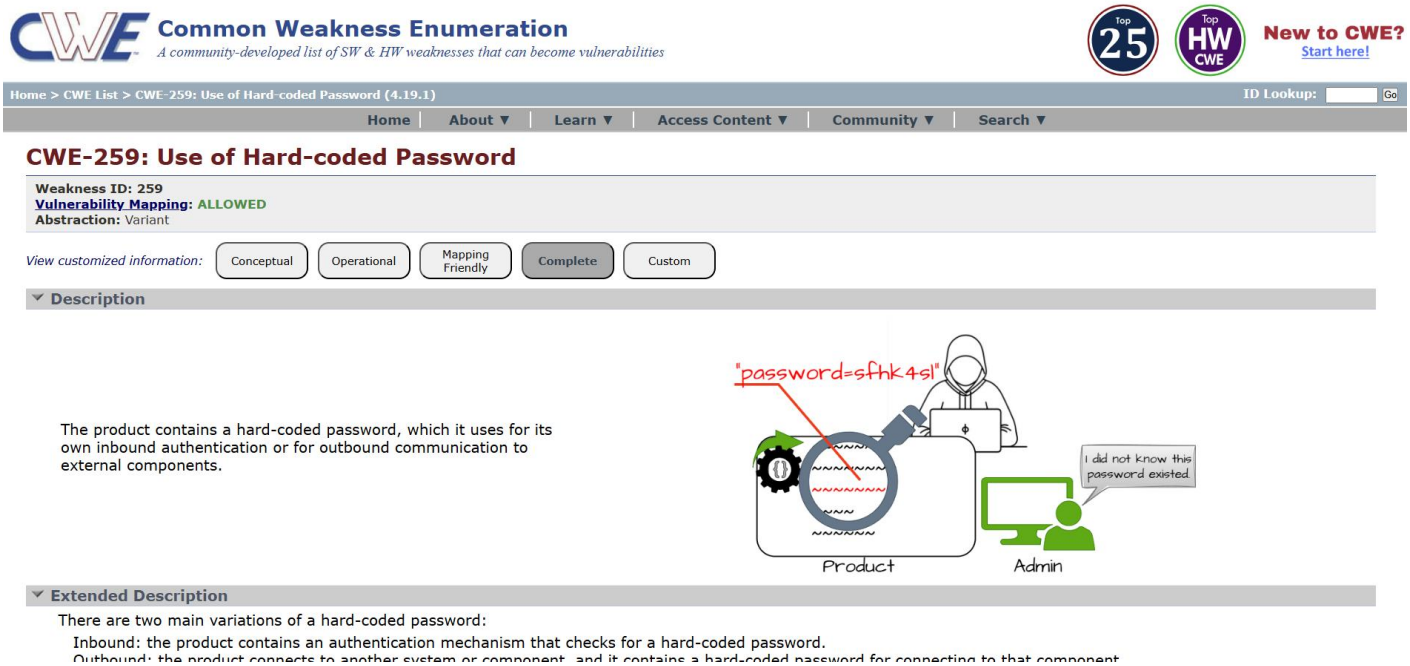
OWASP Top 10 – A07: Identification and Authentication Failures

DESCRIPTION:

Hard-coded passwords are credentials embedded directly in the source code. If attackers obtain the code, they can easily discover these credentials.

BUSINESS IMPACT:

Unauthorized users may gain access to restricted systems or applications, resulting in data breaches and security risks.



8.VULNERABILITY NAME:

Untrusted Search Path

CWE:

CWE-426

OWASP/SANS CATEGORY:

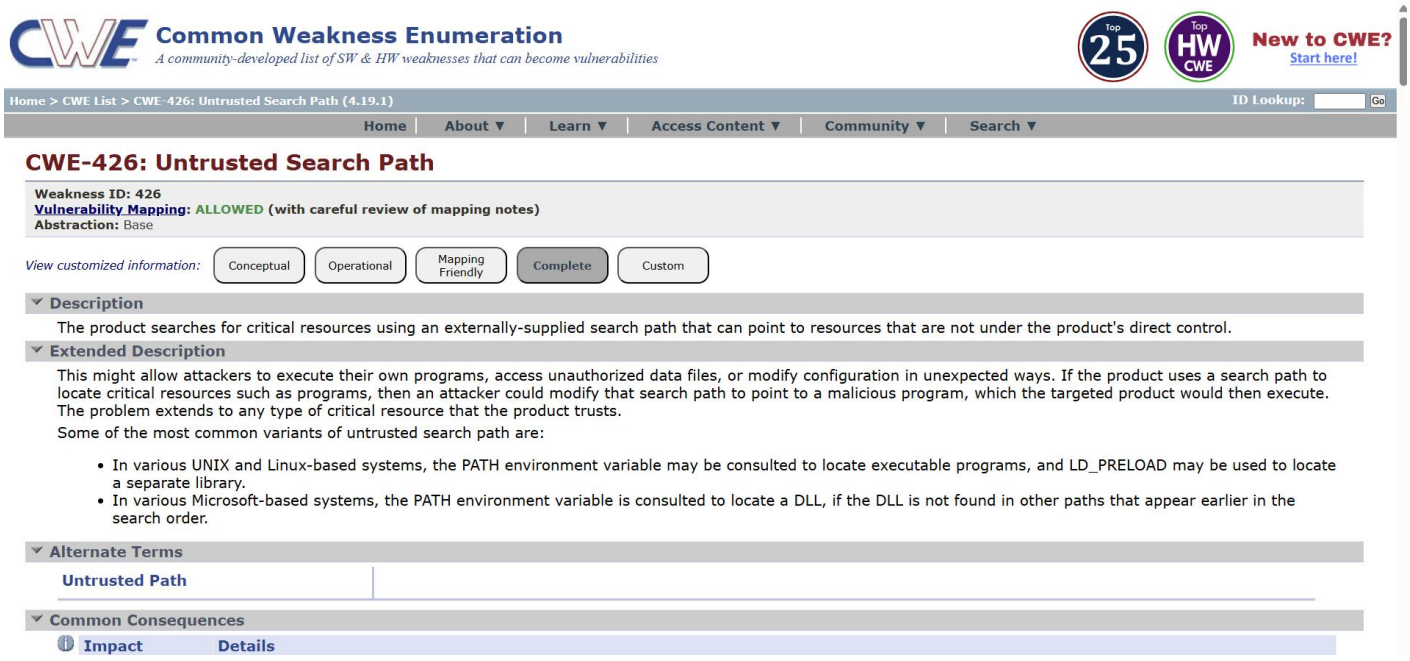
OWASP Top 10 – A08: Software and Data Integrity Failures

DESCRIPTION:

An untrusted search path vulnerability occurs when an application loads resources or executables from an untrusted directory. Attackers may place malicious files in these locations.

BUSINESS IMPACT:

Malicious code may be executed, leading to system compromise, malware infection, or unauthorized control over the application.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-426: Untrusted Search Path (4.19.1) | ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-426: Untrusted Search Path

Weakness ID: 426
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product searches for critical resources using an externally-supplied search path that can point to resources that are not under the product's direct control.

Extended Description

This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.

Some of the most common variants of untrusted search path are:

- In various UNIX and Linux-based systems, the PATH environment variable may be consulted to locate executable programs, and LD_PRELOAD may be used to locate a separate library.
- In various Microsoft-based systems, the PATH environment variable is consulted to locate a DLL, if the DLL is not found in other paths that appear earlier in the search order.

Alternate Terms

Untrusted Path

Common Consequences

Impact | **Details**

9.VULNERABILITY NAME:

Information Exposure Through Logs

CWE:

CWE-532

OWASP/SANS CATEGORY:

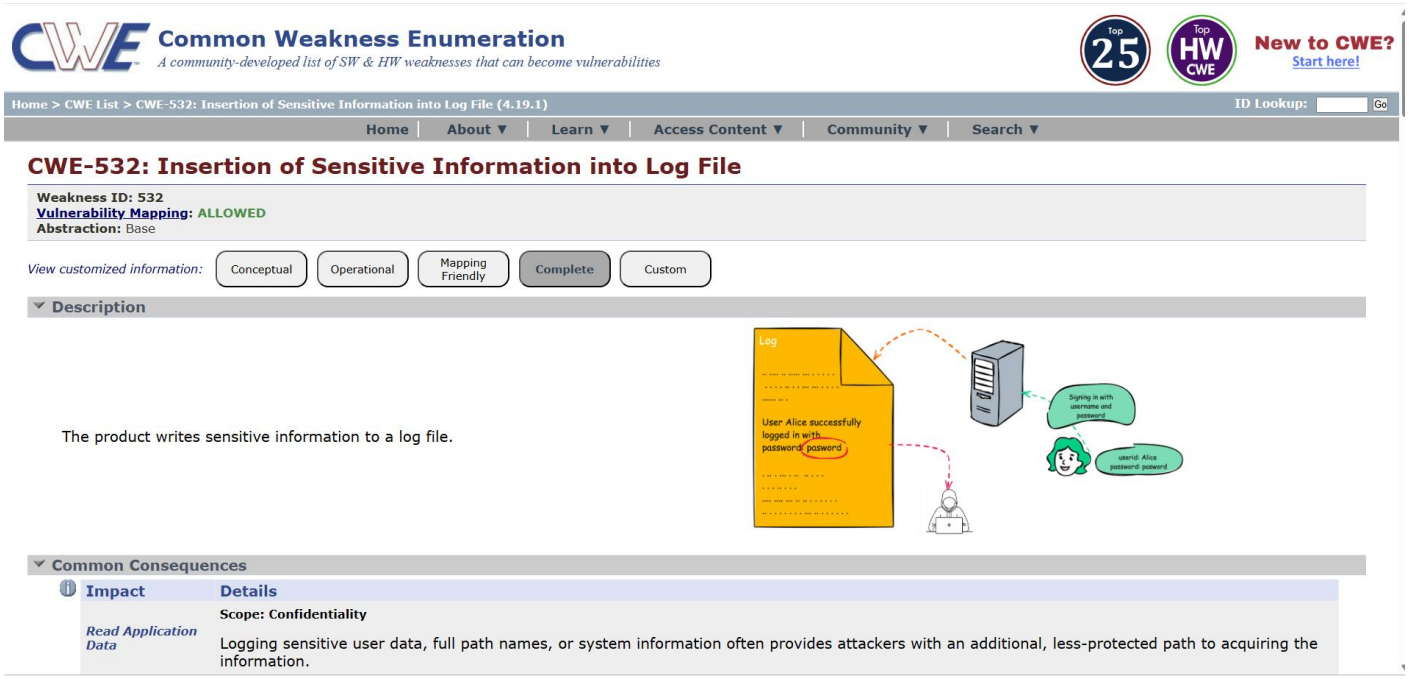
OWASP Top 10 – A09: Security Logging and Monitoring Failures

DESCRIPTION:

Sensitive information such as passwords, tokens, or personal data may be recorded in system log files without proper protection.

BUSINESS IMPACT:

Attackers who gain access to logs may retrieve confidential information and exploit it for further attacks.



10.VULNERABILITY NAME:

Unchecked Return Value

CWE:

CWE-252

OWASP/SANS CATEGORY:

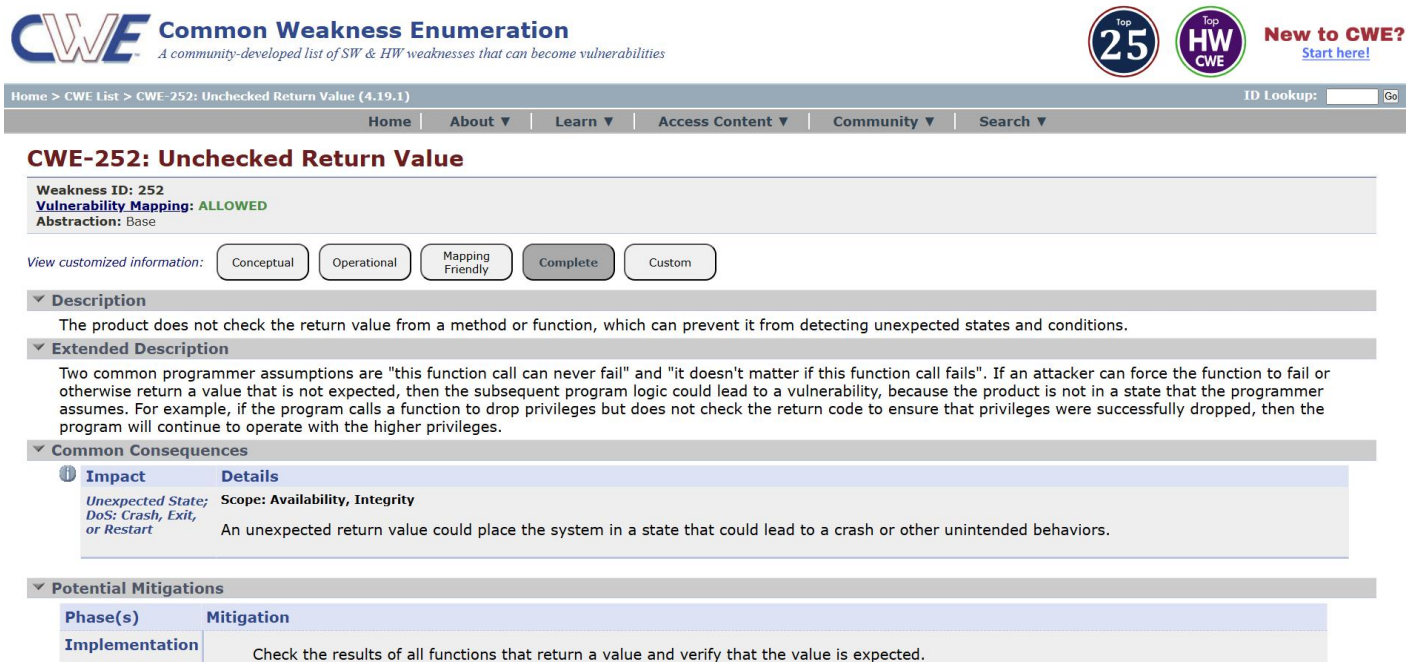
OWASP Top 10 – A10: Server-Side Request Forgery / Improper Error Handling

DESCRIPTION:

Unchecked return values occur when a program does not verify the result returned by a function or system call. This may lead to unexpected behavior or failure to detect errors.

BUSINESS IMPACT:

System errors may go unnoticed, leading to application instability, security weaknesses, or potential exploitation by attackers.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-252: Unchecked Return Value (4.19.1) | ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-252: Unchecked Return Value

Weakness ID: 252
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product does not check the return value from a method or function, which can prevent it from detecting unexpected states and conditions.

Extended Description

Two common programmer assumptions are "this function call can never fail" and "it doesn't matter if this function call fails". If an attacker can force the function to fail or otherwise return a value that is not expected, then the subsequent program logic could lead to a vulnerability, because the product is not in a state that the programmer assumes. For example, if the program calls a function to drop privileges but does not check the return code to ensure that privileges were successfully dropped, then the program will continue to operate with the higher privileges.

Common Consequences

Impact	Details
Unexpected State; DoS: Crash, Exit, or Restart	**Scope: Availability, Integrity** An unexpected return value could place the system in a state that could lead to a crash or other unintended behaviors.

Potential Mitigations

Phase(s)	Mitigation
Implementation	Check the results of all functions that return a value and verify that the value is expected.

STAGE 2

OVERVIEW OF TENABLE NESSUS

Tenable Nessus is one of the most widely used vulnerability assessment tools designed to identify security weaknesses in computer systems, networks, and applications. It is developed by Tenable, Inc. and is used by security professionals to detect vulnerabilities, misconfigurations, outdated software, and potential security risks before attackers can exploit them.

Nessus works by scanning systems and comparing detected configurations against a large vulnerability database. It checks for known vulnerabilities, missing security patches, weak passwords, open ports, and insecure configurations. The tool generates detailed reports that describe each vulnerability, its severity level, potential impact, and recommended remediation steps.

One of the main advantages of Nessus is its large plugin library that continuously updates with new vulnerability checks. It supports scanning for operating systems, web applications, databases, and network devices. Nessus also categorizes vulnerabilities based on risk levels such as **Critical, High, Medium, and Low**, helping administrators prioritize security fixes.

Overall, Tenable Nessus plays a crucial role in proactive cybersecurity by helping organizations detect vulnerabilities early, strengthen their security posture, and protect their infrastructure from cyber threats. It is widely used in security auditing, compliance checking, penetration testing, and vulnerability management programs.

TARGET WEBITE: public-firing-range.appspot.com

TARGET IP ADDRESS: 74.125.30.141

LIST OF VULNERABILITIES :

S.NO	VULNERABILITIES NAME	SEVERITY	PLUGINS ID
1	Cross-site scripting (stored)	Low	Web application Scanner
2	Broken Access Control	Medium	SSH weakHMAC Algorithms Supported
3	SQL Injection	Medium	N/A
4	Broken Authentication	Low	HSTS Missing from Server
5	Insecure Direct Object Reference	Info	Service Detection
6	Security Misconfiguration	Low	SSL cipher Block Chaining cipher suites
7	Cross-Site Request Forgery	Low	HTTP Form- Based Authentication Detected
8	Cleartext Transmission of Sensitive Information	Medium	Cleartext Transmission ofSensitive Information over HTTP
9	ClickJacking	Info	HTML Rendering Layer Vulnerability
10	webserver allows password auto completion	Low	SSL/TLS Diffie-Hellman Modulus<=1024 bits(Logjam)

1. **VULNERABILITY NAME**: Cross site scripting (stored)
SEVERITY: Low
PLUGIN: web application scanner
PORT: N/A
DESCRIPTION: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser.
SOLUTION:
• When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.
• Sanitize and validate all user inputs and ensure that any data displayed on the web page properly encoded. Use output encoding libraries or functions to prevent script injection.
BUSINESS IMPACT: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

2. VULNERABILITYNAME: Broken access control
SEVERITY: medium
PLUGIN:SSH weakHMAC Algorithms supported
PORT: N/A
DESCRIPTION: When access control checks are not applied consistently - or not at all - users are able to access data or perform actions that they should not be allowed to perform. This can lead to a wide range of problems, including information exposures, denial of service, and arbitrary code execution.
SOLUTION:• Ensure that the application has a robust authentication and authorization mechanism in place. Users should be authenticated before any sensitive actions are performed, and authorization checks should be conducted to verify that users have the necessary permissions for the requested operation.
• Ensure that session management is secure. Use strong session IDs, enforce session timeouts, and regenerate session tokens after login. Implement session fixation protection to prevent session hijacking.

BUSINESS IMPACT: wide-ranging business impact, including financial losses, reputation damage, legal consequences, operational disruption, and loss of customer trust. Addressing this vulnerability is crucial to protect the organization and its stakeholders from these potential negative effects.

3. VULNERABILITY NAME: SQL injection
SEVERITY: medium
PLUGIN: N/A (typically would be a security testing tool)
PORT: port 80/443
DESCRIPTION: The product constructs all or part of an SQL command using externally influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component.
SOLUTION:
• If parameterized statements are not feasible, use proper input validation and escaping mechanisms to sanitize user inputs before they are used in SQL queries. This helps prevent malicious code injection.
• Utilize stored procedures for database operations. This can help separate SQL code from application code and reduce the risk of SQL injection.
• Implement strict input validation for all user inputs, ensuring that data adheres to the expected format and structure.
BUSINESS IMPACT: In summary, it is crucial to underscore that CWE-89, known as SQL Injection, can exert a profound and diverse business impact. This encompasses critical facets such as data breaches, financial setbacks, harm to reputation, legal ramifications, and operational turmoil. Hence, the imperative of preventing and remedying SQL injection vulnerabilities cannot be overstated, as it is indispensable for fortifying the security and continuity of an organization's applications and data.

4. VULNERABILITYNAME: Broken authentication
SEVERITY: Low
PLUGIN: HSTS Missing from HTTPS Server (plugin id:142960)
PORT: port 443DESCRIPTION: The system's authorization functionality does not prevent one user from gaining access to another user's data or record by modifying the key value identifying the data.

SOLUTION:

• Implement strong authentication mechanisms, such as multi-factor authentication (MFA) or two-factor authentication (2FA), to ensure that only authorized users can access the system.

• Implement proper authorization controls to ensure that users have the necessary permissions to access specific functions or data. Use role-based access control (RBAC) to manage user privileges effectively.

• Deploy intrusion detection and prevention systems (IDS/IPS) to monitor and block suspicious activities related to authentication and authorization.

BUSINESSIMPACT: To effectively mitigate the business impact of CWE-285, it is imperative that organizations place a strong emphasis on bolstering their authentication and session management practices. This should encompass the adoption of multi-factor authentication, the secure storage of credentials, meticulous session handling, and a commitment to conducting routine security assessments and testing. The rectification of these vulnerabilities stands as a critical imperative, safeguarding sensitive data, user identities, and the organization's overarching security stature and reputation.

5. VULNERABILITYNAME: Insecure Direct object Reference

SEVERITY: Info

PLUGIN:Service Detection (plugin id:22964)

PORT: port 80/443

DESCRIPTION: Insecure Direct Object Reference (IDOR) is a vulnerability that arises when attackers can access or modify objects by manipulating identifiers used in a web application's URLs or parameters. It occurs due to missing access control checks, which fail to verify whether a user should be allowed to access specific data.

SOLUTION:

• Utilize parameterized queries or prepared statements when interacting with databases. This prevents user-controlled input from being executed as SQL code.

• Implement a Content Security Policy to mitigate the risk of cross-site scripting (XSS) attacks, which can be used to execute unauthorized queries.

• Implement account lockout mechanisms that temporarily suspend accounts after a specified number of unauthorized access attempts to prevent brute force attacks.

BUSINESS IMPACT: To effectively mitigate the business impact of CWE-639, organizations must make it a top priority to fortify their access control and authorization mechanisms. This entails the implementation of robust security measures, the regular conduct of comprehensive security assessments, and the deployment of intrusion detection systems to promptly identify and counter unauthorized access attempts. These measures stand as critical in the defence of sensitive data, the overall security of systems, and the preservation of the organization's esteemed reputation.

6. VULNERABILITY NAME: Security MisconfigurationSEVERITY: low
PLUGIN: SSL cipher Block chaining cipher suites
PORT: port 443
DESCRIPTION:

The product specifies permissions for a security-critical resource in a way that allows that resource to be read or modified by unintended actors.

SOLUTION:

• Implement a change management process to document and review all changes to system configurations. This helps prevent unauthorized or unintended alterations.
• Follow security guidelines provided by software and hardware vendors to ensure secure configurations.
• Regularly perform compliance checks against industry standards and regulations to ensure that configurations remain in compliance with security requirements.

BUSINESS IMPACT:

To effectively lessen the business consequences stemming from CWE-732, organizations must place a strong emphasis on bolstering their configuration management practices. This involves the establishment of secure default settings, the routine conduct of comprehensive security evaluations, and the deployment of intrusion detection systems to promptly detect and address misconfigurations and vulnerabilities. These actions are of paramount importance in safeguarding sensitive data, fortifying system security, and preserving the organization's esteemed reputation.

7. VULNERABILITYNAME: Cross-Site Request Forgery
SEVERITY: Low
PLUGIN:HTTP Form-Based Authentication Detected
PORT: port 8080/443
DESCRIPTION: Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a

web application into executing actions of the attacker's choosing. If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring

funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application.

SOLUTION:

• Implement Synchronizer Token Patterns to include unique, randomly generated tokens in each HTTP request and validate these tokens on the server side to verify the authenticity of the request.

• Utilize the Same Site attribute in cookies to restrict their usage to same-site requests only, preventing them from being sent along with cross-site requests and effectively mitigating CSRF attacks.• Utilize framework-specific protections and security features to prevent CSRF attacks. Many modern web frameworks have built-in mechanisms and libraries for handling CSRF vulnerabilities effectively.

• Implement a Content Security Policy to restrict the sources from which various types of content can be loaded. By specifying the trusted sources of content, you can minimize the risk of malicious code execution and reduce the likelihood of successful CSRF attacks.

BUSINESS IMPACT:

To effectively reduce the business repercussions of CWE-352, organizations must prioritize secure coding practices, including the incorporation of anti-CSRF tokens, while also conducting routine security assessments and testing to detect and rectify CSRF vulnerabilities. Furthermore, educating users on secure browsing practices is pivotal in preventing CSRF attacks. These actions are of paramount importance in the protection of data and in upholding the trust of customers and partners.

8. VULNERABILITYNAME: Cleartext Transmission ofSensitive Information

SEVERITY: Medium

PLUGIN: Cleartext Transmission ofSensitive Information overHTTP

PORT: port 443

DESCRIPTION:

The product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a network interface or any link along the channel, such as a router, but they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling security-critical data.

SOLUTION:

• Educate users and employees on secure data handling practices, such as recognizing secure websites (look for "https") and avoiding insecure Wi-Fi networks.

• Employ data masking or redaction techniques to replace sensitive data with placeholders or

cryptographic representations during transmission.

• Implement encryption for data in transit. Use secure encryption protocols such as TLS/SSL to protect sensitive information during transmission.

• Use secure communication protocols for transmitting data, such as HTTPS for web applications, and ensure that the selected protocols are kept up to date.

BUSINESSIMPACT:

To effectively reduce the business consequences associated with CWE-319, organizations must prioritize the adoption of secure data transmission practices. This includes the utilization of encryption and robust, secure protocols. The routine conduct of security assessments and testing is pivotal in pinpointing and remedying vulnerabilities linked to data transmission. Furthermore, the education of users on secure data handling practices plays a vital role in proactively preventing data exposure incidents. These actions are of paramount importance in the protection of sensitive data and in preserving the trust of both customers and partners.

9. VULNERABILITY NAME: ClickjackingSEVERITY: Info

PLUGIN:HTML Rendering Layer Vulnerability

PORT: port 80/443

DESCRIPTION:

The web application does not restrict or incorrectly restricts frame objects or UI layers that belong to another application or domain, which can lead to user confusion about which interface the user is interacting with.

SOLUTION:

• Set the X-Frame-Options HTTP response header to deny or same-origin to control how your site can be framed. This is supported by most modern browsers.

• Utilize frame-busting JavaScript code in web applications to prevent the embedding of your site within malicious frame’s. This code can disrupt clickjacking attempts.

• Implement a Content Security Policy to restrict which domains can embed your site in iframes. This can help prevent unauthorized framing.

• Implement additional security controls to prevent UI redress attacks, such as clickjacking, within your web application.

BUSINESS IMPACT:

To effectively reduce the business consequences resulting from CWE-1021, organizations must prioritize the implementation of protective measures, such as frame-busting code. Simultaneously, educating users on safe browsing practices plays a crucial role in preventing clickjacking incidents. Additionally, routine security assessments and testing are pivotal for identifying and mitigating vulnerabilities associated with clickjacking. These actions are of paramount importance in upholding user trust, ensuring data protection, and safeguarding the organization's reputation.

10. VULNERABILITY NAME: web server allows password auto-completion

SEVERITY: Low

PLUGIN: SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

PORT: port 443

DESCRIPTION: Weaknesses in this category are related to the design and implementation of data confidentiality and integrity. Frequently these deal with the use of encoding techniques, encryption libraries, and hashing algorithms. The weaknesses in this category could lead to a degradation of the quality data if they are not addressed.

SOLUTION:

• Implement secure password storage mechanisms, such as using strong and salted cryptographic hashing algorithms like crypt or script.

• Implement robust encryption key management practices, including secure key storage, key rotation, and access controls.

• Utilize well-established and proven cryptographic libraries and algorithms for encryption and decryption.

BUSINESS IMPACT:

To effectively lessen the business consequences associated with CWE-310, organizations must prioritize the adoption of secure cryptographic practices. This entails ensuring proper password storage and robust encryption key management. Simultaneously, conducting routine security assessments and testing is pivotal in detecting and mitigating vulnerabilities related to cryptographic issues. Striving for compliance with pertinent data protection regulations and industry standards is equally essential. These actions are of paramount importance in the protection of sensitive data, the upholding of user trust, and the preservation of the organization's esteemed reputation.

STAGE 3

TITLE:

Application-specific access policies using Zscaler Private Access (ZPA).

1. Introduction to Application-Specific Access Policies

Application-specific access policies are security rules that allow users to access only the applications they are authorized to use instead of giving access to the entire network. This approach follows the Zero Trust security model where every user request must be verified before granting access. In modern organizations, employees often work remotely and access cloud-based resources, which increases security risks. By implementing application-specific access policies, organizations can restrict access based on user roles, identity, device security, and location. This helps reduce unauthorized access and protects sensitive business applications from cyber threats.

2. Overview of Zscaler Private Access (ZPA)

Zscaler Private Access (ZPA) is a cloud-based Zero Trust Network Access solution developed by Zscaler. It provides secure access to private applications without exposing them directly to the internet. Unlike traditional VPN solutions, ZPA connects users directly to specific applications instead of the entire network. It verifies the user identity, device posture, and security policies before allowing access. This ensures that only authenticated and authorized users can reach internal applications, improving both security and network efficiency.

3. Zero Trust Security Model

The Zero Trust model is a modern cybersecurity approach that assumes no user or device should be trusted by default, even if they are inside the network. Every access request must be authenticated, authorized, and continuously validated before granting permission. ZPA follows this Zero Trust principle by allowing access only to specific applications based on defined policies. This model reduces the risk of lateral movement by attackers and helps organizations maintain strict control over their digital resources.

4. Policy-Based Access Control

Policy-based access control is a method used to manage user access through predefined rules and conditions. These policies can be based on factors such as user identity, department, device type, location, and security compliance. In ZPA, administrators create policies that determine which users can access certain applications. For example, HR employees may only access the HR portal, while finance staff may access financial systems. This approach ensures that users only interact with the applications necessary for their work.

5. Secure Remote Access

Secure remote access is essential for organizations that support remote work or distributed teams. ZPA enables employees to securely connect to internal applications from any location without using a traditional VPN. The system creates a secure encrypted connection between the user and the specific application. This prevents unauthorized network access and protects sensitive data while maintaining productivity for remote workers.

6. Benefits of Application-Specific Access Policies

Implementing application-specific access policies provides several security and operational benefits. It reduces the attack surface by hiding internal applications from the public internet. It also prevents unauthorized users from accessing sensitive systems. Additionally, this approach simplifies network management and improves user experience by providing direct access to required applications. Organizations can also monitor and log user activities, helping detect and respond to security threats quickly.

7. Challenges and Future Improvements

Although application-specific access policies provide strong security benefits, organizations may face challenges such as policy configuration complexity and integration with existing systems. Proper planning and regular monitoring are required to ensure policies are correctly implemented. Future improvements in security technologies, automation, and AI-based threat detection may further enhance the effectiveness of application-specific access systems.

Conclusion :-

Stage 1: Understanding Web Application Testing

Web application testing is the process of examining a web application to identify security vulnerabilities, functional errors, and performance issues before it is deployed or used by real users. The main goal is to ensure that the application is secure, reliable, and works correctly in different environments such as browsers, devices, and networks.

In security-focused web application testing, testers analyze the application to detect vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), authentication issues, and misconfigurations. Tools and manual testing techniques are used to simulate attacks and evaluate how the application responds.

This stage helps organizations:

Identify security weaknesses in the application

Protect sensitive user data from attackers.

Ensure the application follows secure coding practices.

Improve the overall reliability and performance of the system.

Overall, web application testing ensures that the website or web system is safe, functional, and resistant to cyber threats before it is released to users.

Stage 2: Understanding the Nessus Report

A Nessus report generated by Tenable Nessus provides detailed information about security vulnerabilities detected in a system, server, or web application during a vulnerability scan.

The report typically includes:

List of detected vulnerabilities in the scanned target.

Severity levels such as Critical, High, Medium, Low, and Informational.

CVE (Common Vulnerabilities and Exposures) identifiers for known vulnerabilities.

Description of the vulnerability explaining the security risk.

Impact analysis showing how attackers could exploit the issue.

Recommended solutions or patches to fix the vulnerability.

From the Nessus report, we can understand:

Which systems are most vulnerable.

The risk level associated with each vulnerability.

What security updates or configuration changes are required.

This report helps security teams prioritize remediation efforts and strengthen the security posture of the organization.

Stage 3: Understanding SOC / SIEM / QRadar Dashboard

A Security Operations Center (SOC) is a centralized unit responsible for monitoring, detecting, and responding to cybersecurity threats in real time. SOC teams use tools like SIEM (Security Information and Event Management) systems to collect and analyze security data from various sources such as servers, networks, firewalls, and applications.

One widely used SIEM platform is IBM QRadar. The QRadar dashboard provides a visual interface that displays security events, alerts, and threat intelligence data.

The dashboard usually shows:

Real-time security alerts and incidents.

Log data collected from multiple systems.

Network traffic analysis and suspicious activity detection.

Threat intelligence and attack patterns.

Risk scores and offense summaries.

From the SOC/SIEM/QRadar dashboard, security analysts can:

Monitor the overall security status of the network.

Quickly identify potential cyber attacks.

Investigate suspicious activities.

Respond to incidents and mitigate threats efficiently.

This stage helps organizations maintain continuous security monitoring and rapid incident response, which is essential for protecting digital infrastructure.

Future enhancements

Stage 1 – Future Scope of Web Application Testing

Web application testing will continue to evolve as web technologies become more complex and cyber threats increase. In the future, testing methods will increasingly rely on automation and Artificial Intelligence (AI) to detect vulnerabilities faster and more accurately. Advanced testing tools will be able to simulate real-world cyber-attacks and identify security weaknesses before attackers exploit them.

Another important future development is the integration of security testing into DevOps practices, commonly known as DevSecOps, where security testing becomes a continuous part of the software development lifecycle. Cloud-based applications, APIs, and mobile-integrated web systems will also require more advanced testing techniques. Overall, the future scope of web application testing focuses on stronger security, faster vulnerability detection, and automated protection against emerging cyber threats.

Stage 2 – Future Scope of the Testing Process (Based on the Nessus Scan)

The testing process using vulnerability scanning tools will become more intelligent and automated. Tools like Tenable Nessus will integrate with cloud infrastructure and enterprise security systems to provide continuous vulnerability monitoring instead of periodic scans.

In the future, vulnerability management platforms will automatically prioritize risks based on their severity, exploit availability, and business impact. Integration with patch management systems will also allow organizations to automatically apply fixes or security updates. Machine learning techniques will help in identifying unknown vulnerabilities and predicting potential threats before they occur. This will significantly improve the efficiency and effectiveness of cybersecurity testing processes.

Stage 3 – Future Scope of SOC / SIEM

The future of Security Operations Centers (SOC) and Security Information and Event Management (SIEM) systems will focus on automation, artificial intelligence, and faster incident response. Modern SIEM platforms such as IBM QRadar will integrate advanced analytics, threat intelligence, and behavioral analysis to detect complex cyber threats.

Future SOC environments will adopt SOAR (Security Orchestration, Automation, and Response) technologies to automatically respond to certain types of attacks without human intervention. Cloud-based SOC platforms will also enable organizations to monitor distributed networks and remote systems more efficiently. These improvements will help security teams detect threats earlier, reduce response time, and strengthen overall cybersecurity defenses.

Topics Explored

During this activity, the following cybersecurity concepts were explored:

Web application security testing

Vulnerability scanning and assessment

Security misconfiguration detection

Risk classification and severity analysis

Security monitoring and threat detection

Incident analysis using SOC dashboards

Log analysis and event correlation

Security Information and Event Management (SIEM)

Tools Explored:

Tenable Nessus – Used for vulnerability scanning and generating security reports.

Web browsers (for accessing and testing web applications)

What is my IP Address.